

1 End-to-end response growth for positive-depth powers

Source: PrimeTensor/Analysis/Power/Growth.lean

What this file establishes

PrimeTensor/Analysis/Growth.lean replaced the chain rule's transport assumption by a first-order growth condition, and verified that condition for the identity and for constants — the two maps where it is vacuous. This file supplies the first case with content: *every positive-depth power is response-scale controlled*. With it, all three hypotheses the chain rule places on an inner function are discharged for $y \mapsto y^d$, and the file closes by stating the resulting rule.

A power costs refinement. The d -th power is $|d| - 1$ multiplications, each of which spends one level by PrimeTensor/Analysis/Near/Laws.lean, so controlling h^d at level ℓ requires controlling h at level ℓ refined $|d| - 1$ times. Recovering the level afterwards is a *subtraction*, and Depth has none. The first half of the file builds the two transforms that encode the cost without one, and Design note 1.4 is about the identity that makes them fit together.

1.1 Encoding a finite refinement cost

Definition 1.1 (Depth.powerInput and Depth.powerOutput).

$$\begin{aligned} \text{powerInput}(\text{one}, \ell) &= \ell, & \text{powerInput}(\text{succ } d, \ell) &= \text{powerInput}(d, \text{succ } \ell), \\ \text{powerOutput}(\text{one}, \ell) &= \ell, & \text{powerOutput}(\text{succ } d, \text{one}) &= \text{one}, \\ & & \text{powerOutput}(\text{succ } d, \text{succ } \ell) &= \text{powerOutput}(d, \ell). \end{aligned}$$

```
def powerInput : Depth → Depth → Depth
| .one, level => level
| .succ d, level => powerInput d (.succ level)
```

Numerically,

$$|\text{powerInput}(d, \ell)| = |\ell| + |d| - 1, \quad |\text{powerOutput}(d, \ell)| = \max(|\ell| - (|d| - 1), 1).$$

The first adds the power's cost to a requested level; the second removes it, saturating at the coarsest level rather than running off the bottom, since as the docstring says there is no scale below one. Both are defined by recursion and neither mentions a difference: powerInput moves a succ from the power onto the level, and powerOutput cancels succ against succ and stops when the level runs out.

Lemma 1.2 (Depth.powerInput_succ and powerInput_atOrAfter). $\text{powerInput}(d, \text{succ } \ell) = \text{succ powerInput}(d, \ell)$, and $\text{AtOrAfter}(\ell, \text{powerInput}(d, \ell))$.

Proof. Both by structural recursion on d . The first says the two additions commute; the second says adding a cost only refines, and follows from the first by transitivity, one later at a time. $\square$

Theorem 1.3 (Depth.powerInput_output_join). For all d and ℓ ,

$$\text{powerInput}(d, \text{powerOutput}(d, \ell)) = \text{join}(\ell, \text{powerInput}(d, \text{one})).$$

Proof. By simultaneous recursion on d and ℓ ; three of the four cases are rfl. In the remaining case both arguments are successors, and the chain is: unfold both transforms one step; move the exposed `succ` out of `powerInput` by Lemma 1.2; apply the recursive case; observe that `succ` of a join is the join of the successors; and move the `succ` back inside `powerInput`. $\square$

Design note 1.4 (Truncated subtraction, and why the floor is harmless). Numerically the theorem reads

$$(|\ell| - (|d| - 1))^+ + (|d| - 1) = \max(|\ell|, |d|),$$

the standard identity for truncated subtraction: adding back what was truncated recovers the original value, or the truncation threshold, whichever is larger. `powerOutput` is a left adjoint to `powerInput` in the order `AtOrAfter`, and this is the composite of the adjunction on one side.

The identity is exactly what the main theorem needs, and the reason is worth stating. Asked to control the response at level ℓ , the proof offers the response level `powerOutput( $d, \ell$ )`. For that to be justified the perturbation must be controlled at `powerInput( $d, \text{powerOutput}(\mathbf{d}, \ell)$ )`, which by the theorem is not ℓ — it is ℓ joined with the power’s minimum budget `powerInput( $d, \mathbf{one}$ )`, that is, with $|d|$. When ℓ is already at least that fine the join is ℓ and nothing is lost. When it is not, the floor has bitten and more is being asked of the perturbation than the caller supplied.

That is precisely why Theorem 1.11 opens by extracting an anchor beyond which the perturbation is near the pivot at the minimum budget. The convergence hypothesis on h pays for the floor, once, at the start; after that the join is available for free at every stage and the adjunction closes.

Lemma 1.5 (Three facts about `Depth.advance`). *Advancing preserves the order in the base, and*

$$\begin{aligned} \text{advance}(\text{succ } \ell, g) &= \text{succ } \text{advance}(\ell, g), \\ \text{advance}(\text{powerInput}(d, \ell), g) &= \text{advance}(\ell, \text{powerInput}(d, g)). \end{aligned}$$

The three are `advance_base_atOrAfter`, `advance_succ_base` *and* `advance_powerInput_comm`.

Proof. The first two are recursions on the gain. The third is a recursion on d using the first two; numerically both sides are $|\ell| + |g| + |d| - 1$, so it is the commutativity and associativity of adding three costs, proved by moving one `succ` at a time. $\square$

`PrimeTensor/Analysis/Control.lean` proved that advancing respects the order in the *gain*; the first of these is the companion statement in the base, and both are needed because the main proof advances a comparison between two levels rather than between two gains.

Lemma 1.6 (`Depth.join_eq_left_or_right` and `MulReal.scaleNear_join`). *join(a, b) is equal to a or to b . Consequently, if x and y are near at level a and also near at level b , they are near at join(a, b).*

Proof. The first is a recursion on both arguments; being a maximum, the join is whichever argument attains it, and the recursion produces the disjunction. The second then needs no combination argument at all: rewrite along whichever disjunct holds and hand back the corresponding hypothesis. $\square$

Remark 1.7 (A third property of join). `PrimeTensor/Completion.lean` proved join to be an upper bound of its two arguments and remarked that nothing more was proved or needed. Here a third property appears, and it is stronger than it looks: a bound equal to one of the things it bounds

is a *least* upper bound. The file does not say so, and uses the disjunction only to avoid having to combine two nearness facts — which, given that `ScaleNear` has no transitivity, would otherwise not be possible at all.

1.2 Powers on the carrier

Lemma 1.8 (`MulReal.depthPow_mul`). $(ab)^d = a^d b^d$ for every depth d .

Proof. By recursion on d . At one both sides are ab . At `succ d` the claim unfolds to $(ab)(ab)^d = (a a^d)(b b^d)$; the recursive case rewrites the left factor's tail, and the four-factor regrouping of `PrimeTensor/Analysis/Rules.lean` finishes. $\square$

`PrimeTensor/Analysis/Examples.lean` noted that `depthPow` had been given no laws. This is the first of them, and the only one the file needs; $a^m a^n = a^{m+n}$ is still absent, and still unstatable, since `Depth` has no addition.

Theorem 1.9 (`MulReal.scaleNear_depthPow`). *If h is near the pivot at `powerInput(d, ℓ)`, then h^d is near the pivot at ℓ .*

Proof. By recursion on d . At one the power is h and the input level is ℓ , so the hypothesis is the conclusion.

At `succ d` the hypothesis is nearness at `powerInput(d, succ ℓ)`. Weakening it by Lemma 1.2 and `PrimeTensor/Analysis/Control.lean` gives h near the pivot at `succ ℓ`; and the recursive case, applied to the same hypothesis at level `succ ℓ`, gives h^d near the pivot at `succ ℓ`. Two facts at `succ ℓ` multiply to one at ℓ , and $h \cdot h^d = h^{\text{succ } d}$. $\square$

This is the quantitative statement that a power costs refinement, and the budget is exact: $|d| - 1$ multiplications, one level each, is precisely the $|d| - 1$ that `powerInput` adds.

1.3 Response-scale control of a power

Lemma 1.10 (`response_depthPow`). $\text{response}(y \mapsto y^d, x, h)_n = (h_n)^d$.

Proof. The response is $\text{ratio}((x h_n)^d, x^d)$; expand the numerator by Lemma 1.8 into $x^d (h_n)^d$, and the reconstruction identity of `PrimeTensor/Analysis/Derivative.lean` cancels x^d . $\square$

An identity again, as `PrimeTensor/Analysis/Examples.lean` predicted for the whole monomial family.

Theorem 1.11 (`responseScaleControlledAt_depthPow`). *For every x and every depth d , the map $y \mapsto y^d$ is response-scale controlled at x .*

Proof. Let h approach the pivot and let a target gain t be requested. Ask the convergence of h for the level `powerInput(d, one)` — the power's minimum budget — and take the anchor it returns. Offer `powerInput(d, t)` as the response gain.

Beyond the anchor, suppose h_n is near the pivot at level ℓ . It is also near the pivot at the minimum budget, so by Lemma 1.6 it is near at `join(ℓ, powerInput(d, one))`. Offer $\rho = \text{powerOutput}(d, \ell)$ as the response level. Two things must be checked.

The response is near the pivot at ρ . By Theorem 1.3 the joined level just obtained is `powerInput(d, ρ)`, so Theorem 1.9 puts $(h_n)^d$ near the pivot at ρ , and Lemma 1.10 identifies that with the response.

The comparison holds. Again by Theorem 1.3, $\text{powerInput}(d, \rho)$ is a join with ℓ as one argument, so ℓ lies at or before it. Advance both by t , using the base-monotonicity of Lemma 1.5, and then commute the advance past the input transform by the third part of the same lemma:

$$\text{advance}(\text{powerInput}(d, \rho), t) = \text{advance}(\rho, \text{powerInput}(d, t)),$$

whose right-hand side is ρ advanced by the offered response gain. That is exactly the required $\text{AtOrAfter}(\text{advance}(\ell, t), \text{advance}(\rho, r))$. $\square$

Corollary 1.12 (`hasMulDerivativeAt_comp_depthPow`). *If g has derivative D_g at x^d and D_g is scale controlled, then $y \mapsto g(y^d)$ has derivative $\text{comp}(D_g, \text{depthPow}(d))$ at x .*

Proof. Apply the chain rule of `PrimeTensor/Analysis/Growth.lean`. Its three hypotheses on the inner map are now all available: the derivative from `PrimeTensor/Analysis/Examples.lean`, the tangent control from `PrimeTensor/Analysis/Control.lean`, and the response-scale control from Theorem 1.11. $\square$

Remark 1.13 (The first chain rule with content). Both `PrimeTensor/Analysis/Chain.lean` and `PrimeTensor/Analysis/Growth.lean` closed with the observation that the rule could be discharged only for inner maps — the identity, then also the constants — where it says nothing. Corollary 1.12 is the first instance that is not of that kind: the inner map genuinely distorts scale, by $|d| - 1$ levels, and the hypotheses are met anyway because the distortion is finite and exactly accounted. Nothing about g is assumed beyond differentiability and control of its tangent — in particular g needs no response-scale control of its own, the rule requiring that only of the inner factor.

Remark 1.14 (Still no iteration). Applying the corollary twice would require $\text{comp}(D_g, \text{depthPow}(d))$ to be scale controlled, and composition is still not shown to preserve that property — the gap named in `PrimeTensor/Analysis/Chain.lean` and untouched here. Likewise `ResponseScaleControlledAt` is proved for powers, for the identity and for constants, but for no product, inverse or composite of them, so a general monomial $y \mapsto c y^d$ is not covered even though each of its factors is.

Summary of the correspondence

Lean declaration	Kind	Here
<code>Depth.powerInput</code>	definition	Def. 1.1
<code>Depth.powerOutput</code>	definition	Def. 1.1
<code>Depth.powerInput_succ</code>	theorem	Lem. 1.2
<code>Depth.powerInput_atOrAfter</code>	theorem	Lem. 1.2
<code>Depth.powerInput_output_join</code>	theorem	Thm. 1.3
<code>Depth.advance_base_atOrAfter</code>	theorem	Lem. 1.5
<code>Depth.advance_succ_base</code>	theorem	Lem. 1.5
<code>Depth.advance_powerInput_comm</code>	theorem	Lem. 1.5
<code>Depth.join_eq_left_or_right</code>	theorem	Lem. 1.6
<code>MulReal.scaleNear_join</code>	theorem	Lem. 1.6
<code>MulReal.depthPow_mul</code>	theorem	Lem. 1.8
<code>MulReal.scaleNear_depthPow</code>	theorem	Thm. 1.9
<code>response_depthPow</code>	theorem	Lem. 1.10
<code>responseScaleControlledAt_depthPow</code>	theorem	Thm. 1.11
<code>hasMulDerivativeAt_comp_depthPow</code>	theorem	Cor. 1.12

Every declaration of `PrimeTensor/Analysis/Power/Growth.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. Design note 1.4 and Remarks 1.7, 1.13 and 1.14 are stated for the reader and are not declarations of the file; the numerical readings of the two transforms are glosses and are not formalised.